



# Asincronismo

## Parte 2

# Contenido



- ❖ fetch.
- ❖ Promesas en JavaScript.
- ❖ async / await.
- ❖ CORS.



# fetch: peticiones asincrónicas

- ❖ **fetch** es el nombre de una nueva API para JavaScript con la cual se realizan peticiones HTTP asíncronas utilizando **promesas** y de forma que el código sea un poco más sencillo y menos verbose.

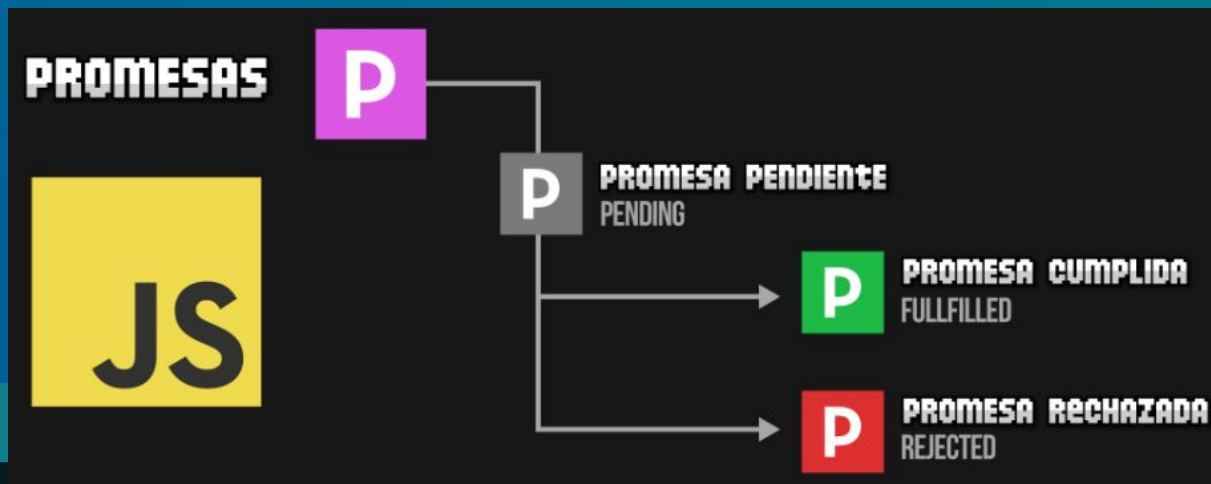
```
fetch("Url")  
.then(response => {  
    //código aquí  
});
```

- ❖ El fetch() retornará una **promesa** que deberá ser aceptada cuando reciba una respuesta y solo será rechazada si hay un fallo.



# Promesas en JavaScript (1/2)

Las promesas se representan mediante un objeto y cada promesa estará en un estado concreto: **pendiente**, **aceptada** o **rechazada**.





# Promesas en JavaScript (2/2)

| Métodos                                                               | Descripción                                                                         |
|-----------------------------------------------------------------------|-------------------------------------------------------------------------------------|
| <code>.then( <b>FUNCTION</b> resolve )</code>                         | Ejecuta la función callback <code>resolve</code> cuando la promesa se cumple.       |
| <code>.catch( <b>FUNCTION</b> reject )</code>                         | Ejecuta la función callback <code>reject</code> cuando la promesa se rechaza.       |
| <code>.then( <b>FUNCTION</b> resolve, <b>FUNCTION</b> reject )</code> | Método equivalente a las dos anteriores en el mismo <code>.then()</code> .          |
| <code>.finally( <b>FUNCTION</b> end )</code>                          | Ejecuta la función callback <code>end</code> tanto si se cumple como si se rechaza. |

```
fetch("Url")  
  .then(response => response.text())  
  .then(data => console.log(data))  
  .catch(err => console.log(err.message))  
  .finally (()=> console.log("terminado..."));
```



# Opciones de fetch

| Campo                                  | Descripción                                                                                                                     |
|----------------------------------------|---------------------------------------------------------------------------------------------------------------------------------|
| <b>STRING</b> <code>method</code>      | Método HTTP de la petición. Por defecto, <code>GET</code> . Otras opciones: <code>HEAD</code> , <code>POST</code> , etc...      |
| <b>OBJECT</b> <code>headers</code>     | Cabeceras HTTP. Por defecto, <code>{}</code> .                                                                                  |
| <b>OBJECT</b> <code>body</code>        | Cuerpo de la petición HTTP. Puede ser de varios tipos: <code>String</code> , <code>FormData</code> , <code>Blob</code> , etc... |
| <b>STRING</b> <code>credentials</code> | Modo de credenciales. Por defecto, <code>omit</code> . Otras opciones: <code>same-origin</code> e <code>include</code> .        |

```
const opciones = {  
  method: "POST",  
  headers: { "Content-Type": "application/json"  
},  
  body: JSON.stringify(data)  
};
```



# El objeto response (1/2)

Encapsula la respuesta que llega desde el servidor y contiene propiedades y métodos.

| Propiedad                 | Descripción                                                                         |
|---------------------------|-------------------------------------------------------------------------------------|
| <b>NUMBER</b> .status     | Código de error HTTP de la respuesta (100-599).                                     |
| <b>STRING</b> .statusText | Texto representativo del código de error HTTP anterior.                             |
| <b>BOOLEAN</b> .ok        | Devuelve <code>true</code> si el código HTTP es <code>200</code> (o empieza por 2). |
| <b>OBJECT</b> .headers    | Cabeceras de la respuesta.                                                          |
| <b>STRING</b> .url        | URL de la petición HTTP.                                                            |



## El objeto response (2/2)

| Método                                     | Descripción                                                                              |
|--------------------------------------------|------------------------------------------------------------------------------------------|
| <b>STRING</b> .text()                      | Devuelve una promesa con el texto plano de la respuesta.                                 |
| <b>OBJECT</b> .json()                      | Idem, pero con un objeto <code>json</code> . Equivale a usar <code>JSON.parse()</code> . |
| <b>OBJECT</b> .blob()                      | Idem, pero con un objeto <code>Blob</code> (binary large object).                        |
| <b>OBJECT</b> .arrayBuffer()               | Idem, pero con un objeto <code>ArrayBuffer</code> (buffer binario puro).                 |
| <b>OBJECT</b> .formData()                  | Idem, pero con un objeto <code>FormData</code> (datos de formulario).                    |
| <b>OBJECT</b> .clone()                     | Crea y devuelve un clon de la instancia en cuestión.                                     |
| <b>OBJECT</b> Response.error()             | Devuelve un nuevo objeto <code>Response</code> con un error de red asociado.             |
| <b>OBJECT</b> Response.redirect(url, code) | Redirige a una <code>url</code> , opcionalmente con un <code>code</code> de error.       |





# Promesas con async / await

- ❖ Usar `async/await` no es más que lo que se denomina **azúcar sintáctico** (utilizar algo visualmente más agradable que por debajo hace la misma tarea).
- ❖ Regla: un `await` solo se puede ejecutar si está dentro de una función definida como `async`.

```
const request = async (url) => {  
  const response = await fetch(url);  
  if( ! response.ok){ throw new Error("error"); }  
  return await response.text();  
}
```

```
const resultado = await request("Url");  
const otro_resultado = await request("Url2");
```



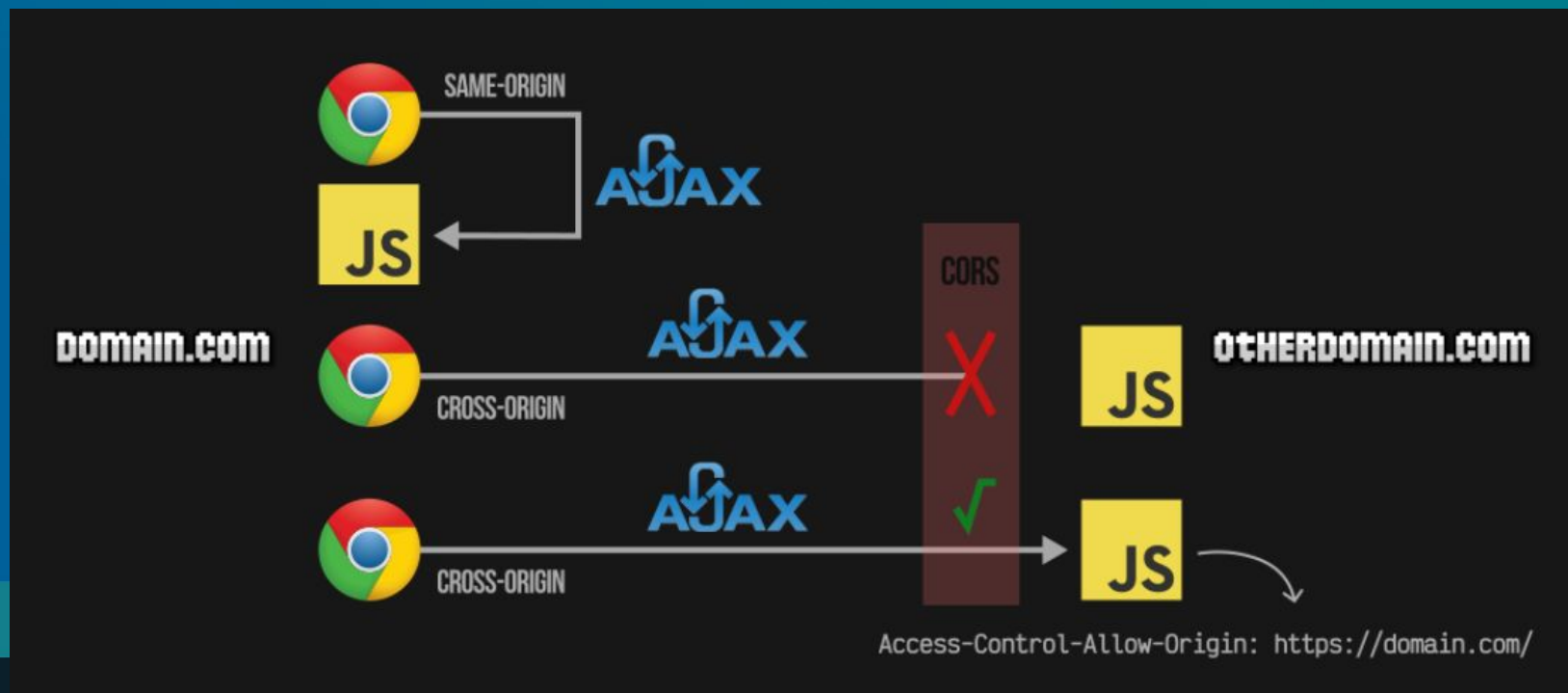
# Ejemplos



# CORS (1/2)

- ❖ CORS (Cross-Origin Resource Sharing) es una política de seguridad que permite controlar las peticiones HTTP **asíncronas** que se pueden realizar desde un navegador a un servidor con un dominio diferente de la página cargada originalmente.
- ❖ Los recursos situados en dominios distintos al de la página actual **no** están permitidos (por defecto) desde peticiones asíncronas (AJAX o fetch).
- ❖ A esto se lo suele denominar **protección de CORS**.

### CORS (1/2)





# Access-Control-Allow-Origin

- ❖ Para evitar la protección de CORS se puede agregar la cabecera **Access-Control-Allow-Origin** en la respuesta de la petición asincrónica, dónde se debe indicar el dominio al que se quiere dar permiso.

```
Access-Control-Allow-Origin: https://domain.com/
```

- ❖ De esta forma el navegador comprobará dicha cabecera y si coinciden con el dominio de origen que realizó la petición, esta se permitirá.

*Nota: con el valor \* se permitirá cualquier dominio.*



# Ejemplos



# Ejercitación